TravelMate: AI-Powered Personalized Travel Itinerary Planner

Project Description:

TravelMate harnesses Generative AI to provide a personalized travel planning experience in one place. The platform includes an AI Itinerary Generator, which creates a day-wise plan from the user's destination, dates, budget and preferences, an Interactive Map, which shows each day's activities with numbered pins and Google Maps directions, and a Budget Tracker, which shows planned cost, remaining budget and daily average. Users can save, edit and delete trips and export the final itinerary as a PDF.

Built with a React + Vite frontend deployed on Vercel and a Python FastAPI backend deployed on Render, TravelMate uses Supabase for authentication and saved trip data, and the Groq API for AI itinerary generation. Secret keys stay on the backend, and the interface was redesigned in seven phases to be modern, responsive and accessible.

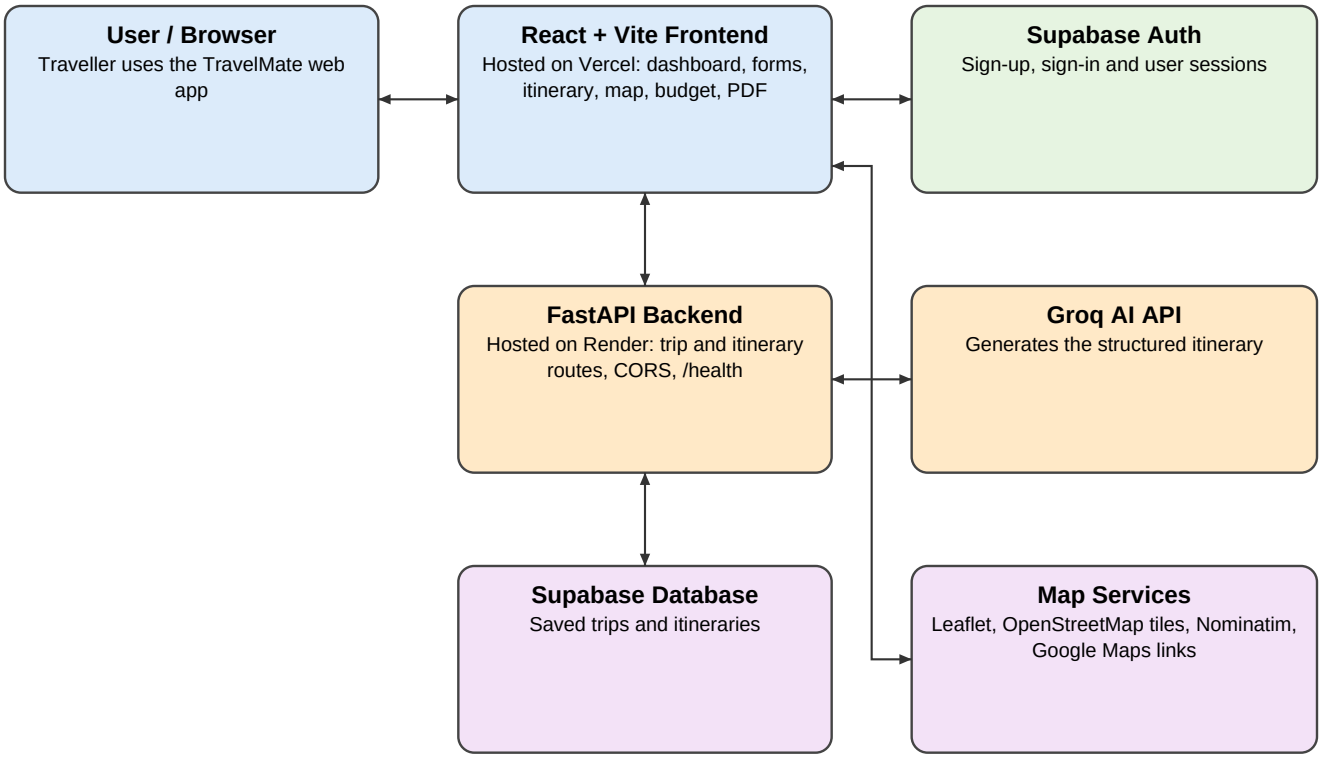
Scenarios

Scenario 1: A student wants a 4-day trip to Kyoto on a limited budget. They sign in, enter the destination, dates, budget and interests, and TravelMate generates a day-wise itinerary with activity times, categories and costs, ready to review in seconds.

Scenario 2: A working professional plans a short weekend getaway. After the itinerary appears, they open the interactive map to check that each day's places are close together and use the multi-stop Google Maps link for directions.

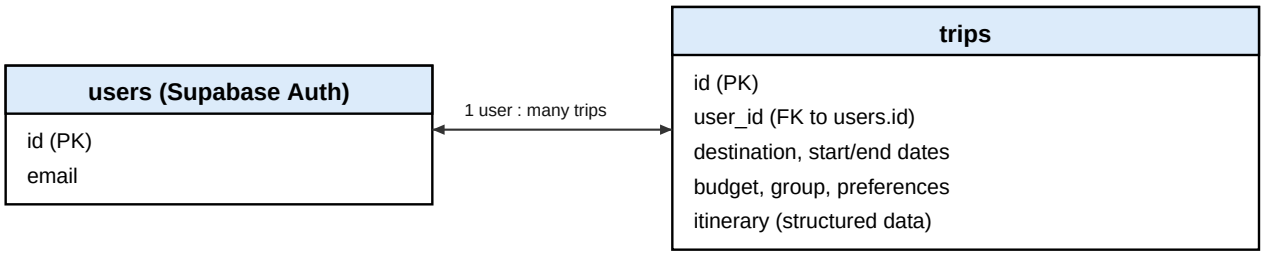
Scenario 3: A family plans a trip together. They check the budget tracker to see the remaining budget and daily average, edit the trip details, regenerate the itinerary and export the final plan as a PDF to share with everyone.

Technical Architecture:



Description: TravelMate uses a modular three-tier architecture. The React + Vite frontend (Vercel) provides the interface and handles maps, budget calculations and PDF export in the browser. The FastAPI backend (Render) receives requests from the frontend, manages trip and itinerary operations and calls the Groq API for AI generation. Supabase provides user authentication and stores saved trips. Leaflet with OpenStreetMap displays the map, Nominatim converts place names to coordinates, and Google Maps links provide navigation.

ER Diagram (logical model): TravelMate stores trips in Supabase. The logical relationship is one user to many trips. Column names below are indicative; confirm them against the actual Supabase table definition before submission.



Pre-requisites:

- **React and Vite Knowledge:** React Documentation
- **FastAPI and Python Proficiency:** FastAPI Documentation
- **Supabase Authentication and Database:** Supabase Documentation
- **Groq API Familiarity:** Groq API Documentation
- **Leaflet Maps:** Leaflet Documentation
- **Version Control with Git and GitHub:** GitHub Documentation
- **Deployment on Vercel and Render:** Vercel Documentation (Render: https://render.com/docs)

Project Workflow:

Milestone 1: Environment Setup and Architecture

- Activity 1.1: Create the Supabase project and the Groq API key and store them securely as environment variables.
- Activity 1.2: Define the architecture covering the frontend, backend, database and AI integration.
- Activity 1.3: Set up the project structure and install frontend and backend dependencies.

Milestone 2: Backend Development

- Activity 2.1: Build the FastAPI application in main.py with CORS and a /health endpoint.
- Activity 2.2: Create trip and itinerary routes, itinerary schemas and the Groq service.

Milestone 3: Frontend Development

- Activity 3.1: Create the design system and the navigation, authentication modal and homepage.
- Activity 3.2: Build the trips dashboard, Create Trip form and Edit Trip modal.

Milestone 4: Itinerary, Map, Budget and PDF

- Activity 4.1: Build the itinerary viewer with day-wise cards and activities.
- Activity 4.2: Add the interactive map, the budget tracker and PDF export.

Milestone 5: Deployment

- Activity 5.1: Deploy the backend on Render and the frontend on Vercel with the correct environment variables.
- Activity 5.2: Verify the deployed application end to end.

Milestone 6: Conclusion

Milestone 1: Environment Setup and Architecture

In this milestone we prepare the services TravelMate depends on: a Supabase project for authentication and data, a Groq API key for AI itinerary generation, and the project structure for the frontend and backend.

Activity 1.1: Create the Supabase Project and the Groq API Key

- **Step 1 - Create a Supabase project:** Sign in at supabase.com, create a new project and note the Project URL and the public (anon) key from the API settings.
- **Step 2 - Create a Groq account:** Visit console.groq.com and sign up.
- **Step 3 - Create an API key:** Open the API Keys page, click Create API Key, give it a name such as travelmate-key and copy it immediately because it is shown only once.

[Insert screenshot]
Supabase project settings (URL and anon key; hide secret values)

[Insert screenshot]
Groq console API Keys page (blur the key)

Step 4 - Add keys as environment variables. The backend reads its secrets from a .env file, and the frontend reads public values from its own .env file:

```
# backend/.env
SUPABASE_URL=your_supabase_project_url
SUPABASE_ANON_KEY=your_supabase_anon_key
GROQ_API_KEY=your_groq_api_key
FRONTEND_URL=http://localhost:5173

# frontend/.env
VITE_SUPABASE_URL=your_supabase_project_url
VITE_SUPABASE_ANON_KEY=your_supabase_anon_key
VITE_API_URL=http://localhost:8000
```

Important - never commit secrets. Add .env to .gitignore. GROQ_API_KEY must only exist on the backend, because every VITE_ variable is included in the browser bundle.

```
.env
```

Activity 1.2: Define the Architecture of the Application

- **Frontend responsibilities:** collect trip details, call the backend, display the itinerary, geocode places for the map, calculate the budget and export the PDF.
- **Backend responsibilities:** receive frontend requests, apply CORS for the frontend origin, manage trip operations and request itineraries from Groq.
- **Data and AI:** Supabase stores users and trips; Groq returns a structured itinerary (destination, summary, total days, days and activities).

Activity 1.3: Set Up the Development Environment

- Install Node.js and Python, and create a Python virtual environment for the backend.
- Backend dependencies are listed in requirements.txt; frontend dependencies are listed in package.json.
- Project structure: frontend (src/components, src/services) and backend (app/routes, app/schemas, app/services).

```
travel-itinerary-planner/
|-- frontend/  (index.html, package.json, vite.config.js, src/...)
|-- backend/   (requirements.txt, app/main.py, routes, schemas, services)
`-- .gitignore
```

Milestone 2: Backend Development

Milestone 2 builds the FastAPI backend that connects the frontend to Supabase and the Groq AI service.

Activity 2.1: Build the FastAPI Application

- **main.py:** creates the FastAPI app, enables CORS for the configured FRONTEND_URL and registers the trip and itinerary routes.
- **database.py:** creates the Supabase client and holds authentication-related backend logic.
- **Health check:** a /health endpoint returns {"status":"ok"} so the hosting platform and the team can confirm the server is running.

Activity 2.2: Create Routes, Schemas and the AI Service

- **routes/trips.py:** trip-related operations such as creating, listing, updating and deleting trips.
- **routes/itineraries.py:** itinerary-related operations that start AI generation for a trip.
- **schemas/itinerary.py:** data models for the itinerary: destination, summary, total days, days and activities (time, category, cost).
- **services/groq_service.py:** sends the trip details to the Groq API and returns the structured itinerary. The API key is read from the backend environment only.

Run the backend locally:

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[Insert screenshot]
Backend running locally / /health response in the browser

Milestone 3: Frontend Development

Milestone 3 focuses on a modern, responsive and accessible interface for TravelMate, delivered in seven redesign phases.

Activity 3.1: Design System, Navigation, Authentication and Homepage

- **Design system (Phases 1-2):** colors, typography (Manrope for headings and DM Sans for body text), spacing, shadows and responsive styles in index.css and App.css.
- **Navigation and authentication (Phase 2):** sticky navbar, mobile navigation and the AuthModal with Sign In / Sign Up tabs, validation, loading states, error and success banners and Escape / backdrop close.
- **Homepage (Phase 3):** hero headline, "Start Planning Free" and "Explore Features" buttons, a sample Kyoto itinerary preview, a features showcase and a 3-step How It Works section.

Activity 3.2: Trips Dashboard and Trip Forms

- **Dashboard (Phase 4):** the "Your Journeys" dashboard with a trip counter, destination-based trip cards, search by destination, sorting (newest, oldest, A-Z, Z-A), View Itinerary, Edit and Delete actions and loading, empty and error states.
- **Create Trip form (Phase 5):** sections for Where & When, Budget & Group and Travel Preferences, with accessible labels and validation.
- **Edit Trip modal (Phase 5):** structured header, scrollable body, sticky footer with Cancel and Save, and Escape key support.

[Insert screenshot]
Homepage hero and Your Journeys dashboard

Milestone 4: Itinerary, Map, Budget and PDF

Activity 4.1: Itinerary Viewer (ItineraryModal.jsx)

- Shows destination information, a trip summary and day-wise cards.
- Activities can be expanded or collapsed and show time, category and cost when available.
- Each activity has a Google Maps link, and each day offers multi-stop Google Maps directions.
- Actions: PDF export, regenerate and close.

Activity 4.2: Map, Budget Tracker and PDF Export

- **Interactive map (DayMap.jsx):** Leaflet with OpenStreetMap tiles and numbered pins per day, a show/hide option, Nominatim geocoding with a coordinate cache and rate limiting, and a fallback when a place cannot be found.
- **Budget tracker:** planned budget, total activity cost, remaining budget, daily average, day-wise cost badges, a progress bar and within-budget / over-budget states. Missing or invalid costs are handled safely.
- **PDF export (pdfExport.js):** jsPDF creates a multi-page A4 PDF with itinerary details, budget and day information, a loading indicator, success feedback and a branded footer.

Planned costs are estimates and do not represent real spending. Map pins depend on valid place names.

Milestone 5: Deployment

In Milestone 5 the backend is deployed on Render and the frontend on Vercel, both connected through environment variables.

Activity 5.1: Deploy the Backend on Render

- Connect the GitHub repository to a new Render web service.
- Set the start command used in the successful deployment:

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

- Add the environment variables SUPABASE_URL, SUPABASE_ANON_KEY, GROQ_API_KEY and FRONTEND_URL (the production frontend origin, used for CORS).
- After deployment the logs showed the FastAPI server starting, and /health returned {"status":"ok"}.

Backend URL: https://personalized-travel-itinerary-planner-k0ey.onrender.com

Activity 5.2: Deploy the Frontend on Vercel

- Import the repository in Vercel and set the root directory to frontend.
- Add VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY and VITE_API_URL (the Render backend base URL), then redeploy. Missing VITE_ variables previously caused the error "supabaseUrl is required".

Frontend URL: https://personalized-travel-itinerary-planner-4f55ulcf3.vercel.app

Source code: https://github.com/harshvardhan4418-crypto/personalized-travel-itinerary-planner

[Insert screenshot]
Vercel deployment dashboard and Render service logs

Activity 5.3: Verification Checklist

The following live checks still need to be completed and recorded on the production deployment:

- Sign up and sign in with Supabase Auth.
- Create, edit and delete a trip.
- Generate an AI itinerary through the backend.
- Open the map and the Google Maps links.
- Download the itinerary PDF.

Exploring the Website's Web Pages:

Home Page:

[Insert screenshot]
TravelMate homepage with hero, features and How It Works

Description: The homepage presents the TravelMate compass branding, a sticky navigation bar and the headline "Design Your Dream Journey in Seconds." Call-to-action buttons lead to planning or the features, a sample Kyoto itinerary shows what users will get, and a 3-step How It Works section explains the flow.

Authentication Modal:

[Insert screenshot]
Sign In / Sign Up modal

Description: A modal with Sign In and Sign Up tabs, input validation, loading states and error or success banners. It closes with the Escape key or a backdrop click, and a signed-in user sees an avatar chip with Sign Out.

Trips Dashboard and Create Trip:

[Insert screenshot]
Your Journeys dashboard and Create Trip form

Description: The "Your Journeys" dashboard lists trips as cards with search and sorting, and View Itinerary, Edit and Delete actions. The Create Trip form collects destination, dates, budget, group and travel preferences in clearly separated sections.

Itinerary, Map and Budget:

[Insert screenshot]
Itinerary modal with day cards, map and budget tracker

Description: The itinerary modal shows the summary and day-wise activity cards, an interactive map with numbered pins, a budget tracker with remaining budget and a progress bar, Google Maps links and PDF export, regenerate and close actions.

Conclusion:

TravelMate is a full-stack AI-powered travel planning web application built with React, Vite, FastAPI, Supabase and Groq. It brings personalized itinerary generation, trip management, an interactive map, budget tracking and PDF export into one place, so travellers no longer need to combine several websites. The project covered environment setup, backend and frontend development, a seven-phase interface redesign and deployment on Vercel and Render, with the code maintained on GitHub.

Looking ahead, TravelMate can be extended with weather information, shareable itineraries, multi-currency budgets, collaborative editing and offline access. The most important next step is to verify the full production workflow, including login, saved trips, AI generation and PDF download, and to review Supabase Row Level Security and backend authorization. With these in place and continuous prompt and interface improvements, TravelMate can grow from a trip planner into a complete travel companion.